Penetration Test Findings Report

Project 3 — Purple-Team Web Application Attack & Defense

Engagement	Internal web-application penetration test (home lab)
Target	ubuntu-dvwa · 192.168.64.8
Attacker	Kali Linux · 192.168.64.7
Author	Sawhard Poojari
Role	SOC Analyst / Purple Team
Date	26 October 2026
Classification	Confidential — Lab Use Only

1. Executive Summary

This report documents a full-chain penetration test conducted against a deliberately vulnerable web server (DVWA on Ubuntu) inside an isolated home lab. The objective was to emulate a realistic external attacker progressing from reconnaissance to full remote command execution, so that the corresponding blue-team detection coverage could be measured. The defensive analysis is covered in a companion document, the SOC Detection & Incident Response Report.

Working from a single attacking host, I was able to move through the entire attack chain and achieve remote code execution on the target as the web-server user. Every objective below was met:

- **Reconnaissance —** Identified open services (SSH, HTTP) and their exact versions through network reconnaissance.
- **Credential access —** Attempted SSH credential brute force (contained by an automated firewall block, see §4.2).
- **Injection —** Exploited SQL injection to extract all application usernames and password hashes.
- **Web shell —** Uploaded a PHP web shell through an unrestricted file-upload flaw.
- **Remote code execution —** Executed arbitrary system commands and read sensitive files (/etc/passwd).

Overall risk: Critical

The chained vulnerabilities allow an unauthenticated attacker to achieve remote command execution and read arbitrary files on the host. SQL injection alone exposes every credential in the application database. These are intentionally present in DVWA, but the same classes of flaw are common in real applications and are rated Critical accordingly.

Scope note: this was an authorised test against a lab system I built and own. DVWA (Damn Vulnerable Web Application) is intentionally insecure and was selected specifically to exercise detection engineering. Industry-standard offensive tools (Nmap, Medusa, curl) were used; the lab design, target build, attack sequencing, and all analysis are my own work. See the Methodology & Authorship note at the end of this report.

2. Scope & Environment

Parameter	Value
Target host	ubuntu-dvwa (hostname: spector)
Target IP	192.168.64.8
Operating system	Ubuntu Linux, kernel 7.0.0-30-generic (aarch64)
Exposed services	22/tcp SSH (OpenSSH 10.2p1), 80/tcp HTTP (Apache 2.4.66)
Web application	DVWA — Damn Vulnerable Web Application, security level: Low
Attacker host	Kali Linux — 192.168.64.7
Network	Isolated host-only lab network, 192.168.64.0/24
Rules of engagement	Full attack permitted; no production systems; lab owned by tester

3. Findings Summary

Five findings were confirmed during the test. Severity reflects real-world impact of each flaw class; CWE references are included for traceability.

#	Finding	Severity	CWE
F-1	SQL Injection (credential & hash disclosure)	**Critical**	CWE-89
F-2	Unrestricted file upload → web shell	**Critical**	CWE-434
F-3	Remote command execution via web shell	**Critical**	CWE-78
F-4	Weak SSH credential policy / brute-forceable login	**High**	CWE-307
F-5	Information disclosure via service banners	**Low**	CWE-200

4. Attack Narrative (Kill Chain)

The engagement followed the standard attacker progression. Each phase is mapped to the relevant MITRE ATT&CK technique and evidenced with output captured from the attacking host.

4.1 Reconnaissance — MITRE T1595 / T1046

The test began with network reconnaissance to identify live services on the target. A TCP SYN scan confirmed two open ports; a service/version scan then fingerprinted the exact software running on each.

```
nmap -sS -p 1-1000 192.168.64.8
nmap -sV 192.168.64.8
```

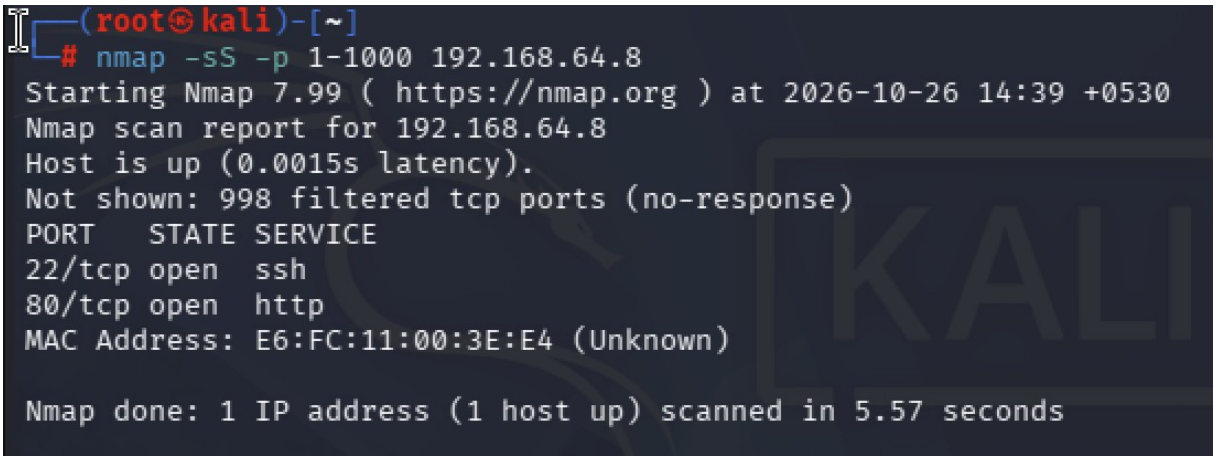


Figure 4.1a — SYN scan: ports 22 (SSH) and 80 (HTTP) open on the target.

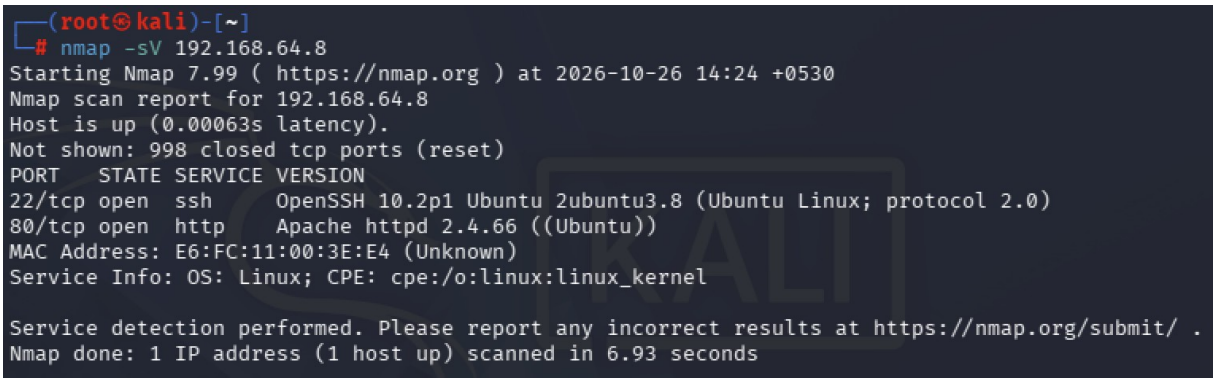


Figure 4.1b — Version scan: OpenSSH 10.2p1 and Apache 2.4.66 fingerprinted. Banners disclose exact versions (Finding F-5).

Result: two attack surfaces identified — an SSH service to target for credential access, and an Apache web server hosting DVWA. The version banners (F-5) hand an attacker the information needed to search for version-specific exploits.

4.2 SSH Credential Brute Force — MITRE T1110

With SSH exposed, I attempted an online password-guessing attack against the admin account using Medusa and a small wordlist of common passwords.

```
medusa -h 192.168.64.8 -u admin -P /tmp/pw.txt -M ssh -t 4
```

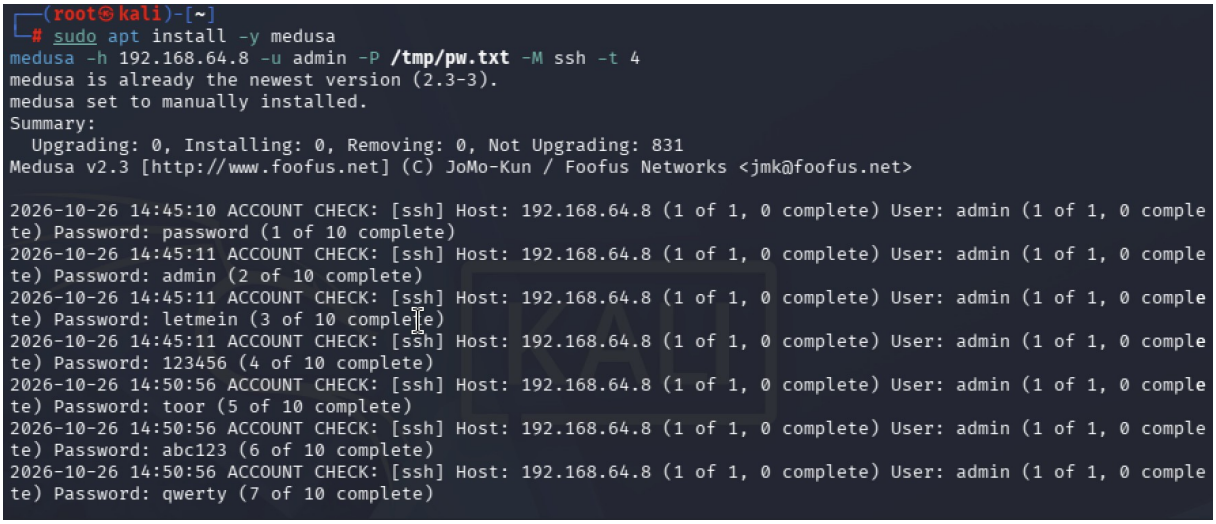


Figure 4.2 — Medusa iterating common passwords against SSH. Note the ~5-minute gap after password 4: the target's automated defence blocked the attacker mid-run (see Finding F-4 and the SOC report §Active Response).

> **Defence observed during the attack**
> This attack was actively disrupted. After four attempts the attacking IP was firewall-blocked by the target's Wazuh Active Response, forcing a ~5-minute pause before the run could continue. The brute force did not succeed. This is the purple-team value of the exercise: the attack and the defensive reaction were observed together.

4.3 SQL Injection — MITRE T1190 (Finding F-1)

The DVWA SQL injection entry point failed to sanitise user input, allowing the query logic to be manipulated. A boolean payload first confirmed the flaw by returning all users; a UNION-based payload then extracted the username and password-hash columns directly from the users table.

```
Payload 1 (confirm):   1' OR '1'='1
Payload 2 (extract):   1' UNION SELECT user, password FROM users-- -
```

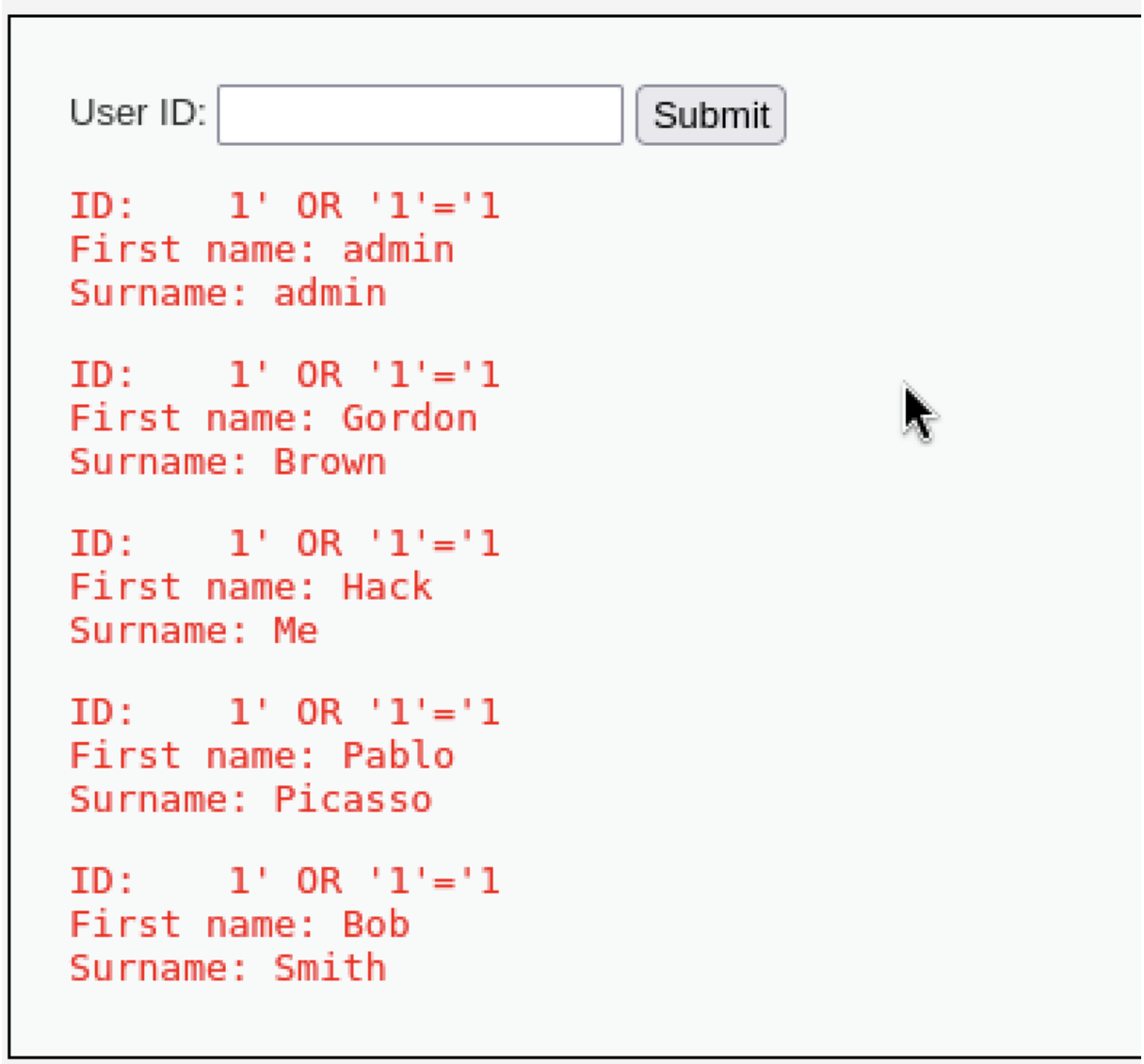


Figure 4.3a — Boolean payload returns every application user, confirming injection.

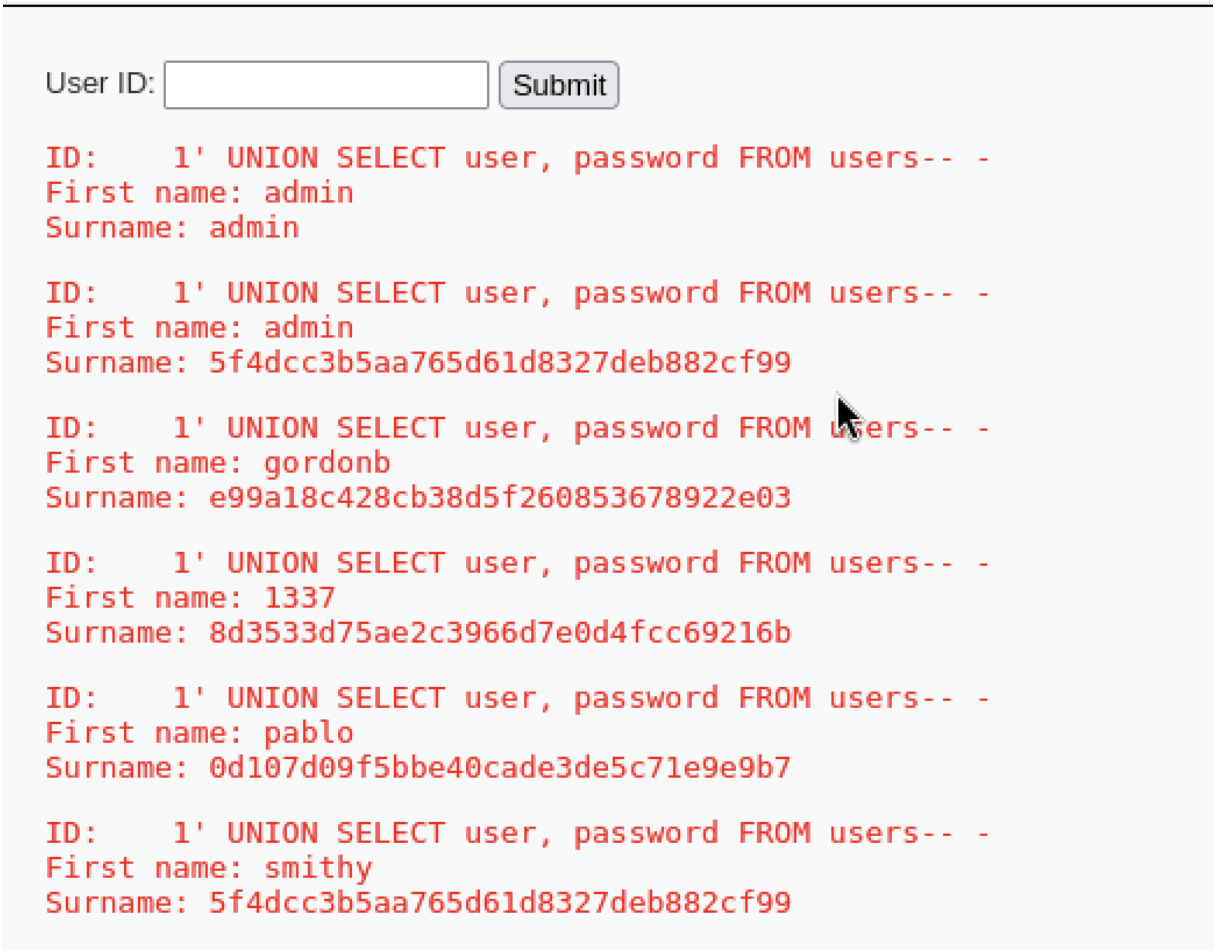


Figure 4.3b — UNION payload dumps usernames and MD5 password hashes directly from the database (e.g. admin : 5f4dcc3b5aa765d61d8327deb882cf99 = "password").

Impact: complete disclosure of the application's credential store. The hashes are unsalted MD5 and crack instantly against any wordlist — the admin hash is literally the word "password". This single flaw compromises every account in the application.

4.4 Web Shell Upload — MITRE T1505.003 (Finding F-2)

The DVWA file-upload feature did not validate file type or content, so a PHP web shell could be uploaded in place of an image. The shell is a one-line command-execution script:

```
<?php system($_GET["cmd"]); ?>
```

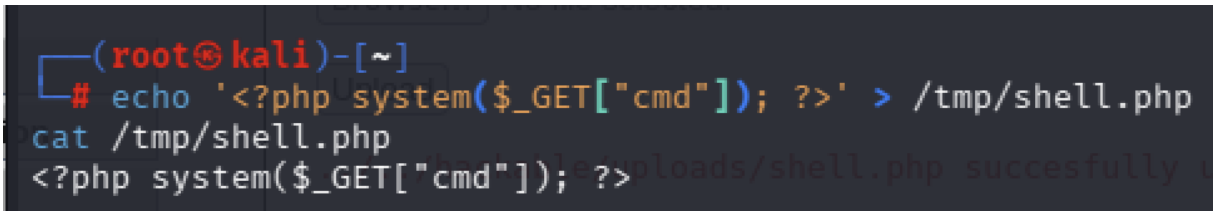


Figure 4.4a — The one-line PHP web shell created on the attacker host.

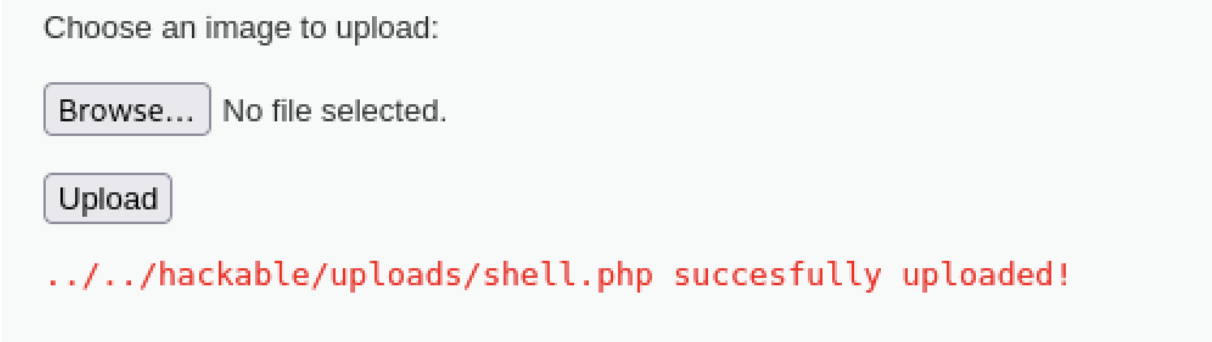


Figure 4.4b — DVWA accepts the .php file and confirms the upload path under /hackable/uploads/.

Result: an attacker-controlled script now resides in a web-accessible directory, ready to execute commands on demand. This upload was detected in real time by File Integrity Monitoring on the defence side.

4.5 Remote Command Execution — MITRE T1059 (Finding F-3)

Requesting the uploaded shell with a cmd parameter executes arbitrary commands on the server. I confirmed the privilege level, enumerated the host, and read a sensitive system file.

```
curl "http://192.168.64.8/dvwa/hackable/uploads/shell.php?cmd=id"
curl ".../shell.php?cmd=whoami"
curl ".../shell.php?cmd=uname%20-a"
curl ".../shell.php?cmd=cat%20/etc/passwd"
```



Figure 4.5 — Remote command execution confirmed: code runs as uid=33(www-data); host and kernel disclosed; /etc/passwd read remotely.

Impact: full remote command execution as the web-server user (www-data). The attacker can read application source and configuration (including DB credentials), enumerate the host, and use this foothold to attempt privilege escalation to root. This is the terminal objective of the kill chain.

5. Detailed Findings & Remediation

F-1 SQL Injection

Severity	Critical
CWE	CWE-89
MITRE ATT&CK	T1190 — Exploit Public-Facing Application

Description. User-supplied input is concatenated directly into a SQL query without parameterisation or validation, allowing an attacker to alter the query and read arbitrary data from the database.

Impact. Full disclosure of the application credential store (usernames + unsalted MD5 hashes). Depending on DB permissions, injection can also allow data modification, authentication bypass, or further host access.

Remediation. Use parameterised queries / prepared statements for all database access. Apply input validation and least-privilege database accounts. Store passwords with a strong adaptive hash (bcrypt/argon2) and a unique salt per user.

F-2 Unrestricted File Upload

Severity	Critical
CWE	CWE-434
MITRE ATT&CK	T1505.003 — Web Shell

Description. The upload feature accepts arbitrary file types and stores them in a web-accessible, executable directory, allowing a server-side script (PHP web shell) to be uploaded in place of an image.

Impact. Attacker gains a persistent, on-demand command-execution channel on the server.

Remediation. Validate file type by content (magic bytes), not extension; store uploads outside the web root or in a directory where script execution is disabled; rename files to random identifiers; serve them with a safe Content-Type and never execute them.

F-3 Remote Command Execution

Severity	Critical
CWE	CWE-78
MITRE ATT&CK	T1059 — Command & Scripting Interpreter

Description. The uploaded web shell passes attacker input to the system() function, executing arbitrary OS commands as the web-server user.

Impact. Full control of the web-server process context (www-data): reading application/config files, host enumeration, and a staging point for privilege escalation to root.

Remediation. Primarily resolved by preventing the upload (F-2). Additionally: run the web server under a constrained account, disable dangerous PHP functions (system, exec, passthru) via disable_functions, and deploy host command-auditing (auditd) to detect any execution that does occur.

F-4 Weak SSH Credential Policy

Severity	High

CWE	CWE-307
MITRE ATT&CK	T1110 — Brute Force

Description. SSH permits password authentication for a predictable account name with no lockout at the application layer, making online password guessing viable.

Impact. Without compensating controls, an attacker could eventually guess valid credentials and gain interactive shell access. In this test the attempt was contained by automated firewall blocking.

Remediation. Enforce key-based authentication and disable password login; use fail2ban or the demonstrated Wazuh Active Response for automatic blocking; rename/disable default accounts; enforce a strong password policy and MFA where interactive login is required.

F-5 Service Banner Information Disclosure

Severity	Low
CWE	CWE-200
MITRE ATT&CK	T1592 — Gather Victim Host Information

Description. SSH and HTTP services return precise version banners, letting an attacker fingerprint software and target version-specific exploits.

Impact. Accelerates reconnaissance and exploit selection. Low severity on its own.

Remediation. Suppress or minimise version banners where supported; keep all services patched so disclosed versions carry no known-exploit risk; place services behind a reverse proxy that normalises response headers.

6. MITRE ATT&CK Mapping

Tactic	Technique	ID	Phase in this test
Reconnaissance	Active Scanning / Network Service Discovery	T1595 / T1046	§4.1 Nmap
Credential Access	Brute Force	T1110	§4.2 Medusa SSH
Initial Access	Exploit Public-Facing Application	T1190	§4.3 SQL injection
Persistence	Server Software Component: Web Shell	T1505.003	§4.4 Shell upload
Execution	Command & Scripting Interpreter	T1059	§4.5 RCE

7. Conclusion

The target was fully compromised through a realistic, chained attack path: reconnaissance identified the attack surface, SQL injection exposed the credential store, an unrestricted upload planted a web shell, and that shell delivered remote command execution. Each flaw is individually serious; chained, they represent a complete breach of the host.

The highest-value remediations are parameterised queries (F-1) and strict upload validation (F-2) — together they break the chain before it reaches command execution. Equally important, this test was

run as a purple-team exercise: the defensive side detected every post-reconnaissance stage and automatically blocked the brute force in real time. Those results are documented in the companion SOC Detection & Incident Response Report.

Methodology & Authorship Note

This engagement was performed in an isolated lab that I designed and built. The offensive tooling (Nmap, Medusa, curl) and the target application (DVWA) are standard, publicly available software. What this report represents as my own work is the end-to-end engineering and judgement: building and networking the lab, standing up the target and the monitored agent, sequencing the attack to emulate a realistic adversary, interpreting each result, and mapping findings to impact, CWE, MITRE, and remediation. Where a control interfered with the attack (the automated SSH block), I have reported it honestly rather than editing it out, because the interaction between attack and defence is the point of the exercise.